Author

P A Athithiya
23F3003503
23f3003503@ds.study.iitm.ac.in
A passionate final year student of B.Tech ECE and currently in diploma in DS

AI/LLM usage

AI/LLM tools were used to a limited extent for requirement clarification, debugging assistance, and code review. The overall AI-assisted contribution was approximately 10 to 12%. All application logic, integration, testing, and project-specific implementation were reviewed and carried out as part of the development process.

Description

The Trekking Management Application is a multi-user web application for managing treks, staff, trekkers, and bookings. It provides separate role-based dashboards for administrators, trek staff, and trekkers with controlled access to their respective functionalities.

Technologies used

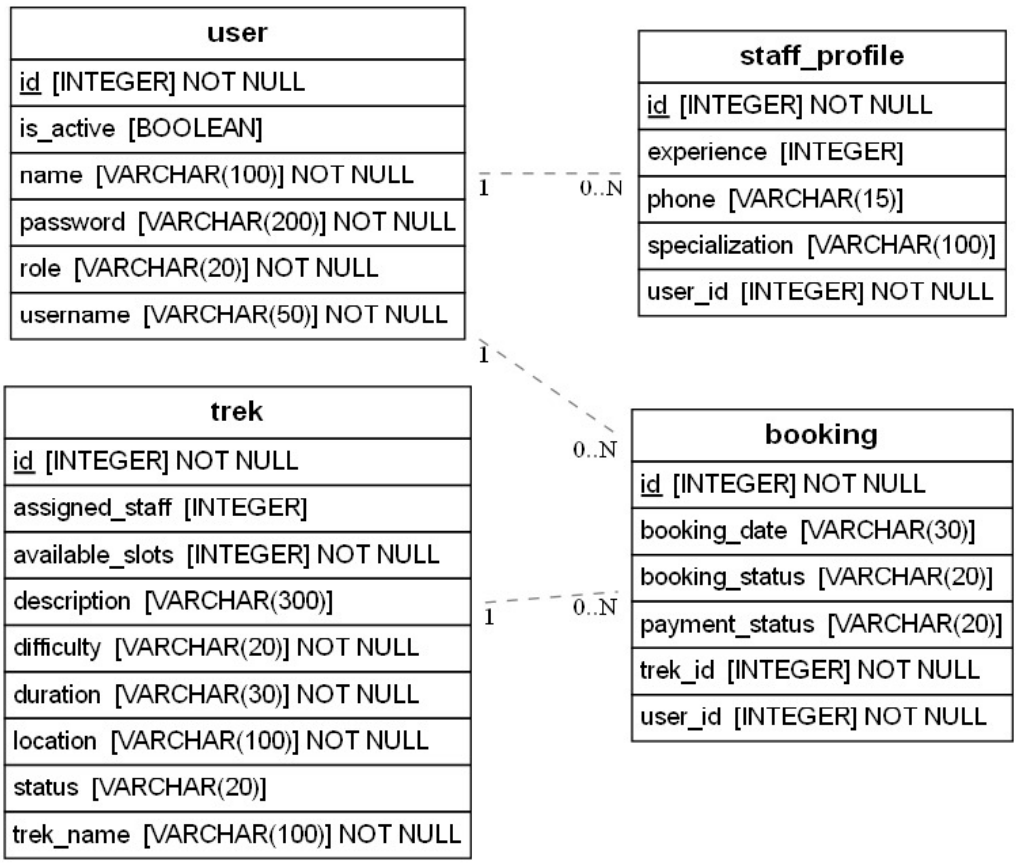
Python, Flask, Flask-SQLAlchemy, SQLite, Jinja2, HTML, Bootstrap 5
Flask is used for application routing and backend logic, while Flask-SQLAlchemy provides ORM-based database operations. SQLite provides lightweight persistent storage, Jinja2 supports dynamic page rendering, and Bootstrap is used to build a responsive user interface. Git and GitHub are used for version control.

DB Schema Design

The application contains four main database models:

Model	Important Columns
User	id (PK), name, username (Unique), password, role, is_active
Trek	id (PK), trek_name, location, difficulty, duration, available_slots, assigned_staff, status, description
Booking	id (PK), user_id (FK), trek_id (FK), booking_status, booking_date, payment_status
StaffProfile	id (PK), user_id (FK, Unique), phone, experience, specialization

The User model stores common authentication information for admin, staff, and trekkers. Staff-specific information is maintained separately in StaffProfile. The Booking model connects users and treks using foreign keys, enabling booking and participant history. This structure reduces duplication and supports role-based application functionality.



Architecture and Features

The application follows a Flask-based architecture. Application routes and controller logic are defined in app.py, database models are maintained in models.py, and Jinja2 templates are stored in the templates directory. SQLite is used for persistent data storage. Session-based authentication and role checks restrict access to administrator, staff, and trekker features. The administrator can manage treks and staff, assign staff to treks, search records, activate or deactivate users, and view booking history. Staff members can access only assigned treks, update slots and trek status, view participants, and manage booking status. Trekkers can register, log in, update their profile, view treks, book available treks, and view booking history. Duplicate bookings and overbooking are prevented, slots are reduced after successful bookings, and a trek is closed when no slots remain.

Video

https://drive.google.com/file/d/1ms-aJGDAgBjXp4zKk-kfamK_c_xWsTZz/view?usp=drive_link